

从计算机专业学生到 Agent 开发工程师

可上岗实战教程 · 16 周路线图 · 项目 · 评测 · Prompt 库

适用对象：计算机科学与技术本科生及转型开发者

从计算机专业学生到 Agent 开发工程师：可上岗实战教程

版本：2026-08 适用对象：计算机科学与技术本科生、掌握基础编程但缺少大模型工程经验的求职者
学习建议：16 周，每周 18-25 小时。若基础薄弱，可扩展到 24 周。
目标岗位：Agent 开发工程师、LLM 应用工程师、AI 应用后端工程师、RAG 工程师。

0. 先说结论：什么叫“达到上岗”

本教程把“可上岗”定义为能够在指导下完成一个中小型 Agent 功能从需求澄清到上线维护的完整闭环，而不是只会调用聊天接口。结业时你应当能：

1. 用 Python、FastAPI、Pydantic、异步 I/O 编写可维护服务。
2. 解释 token、上下文窗口、采样、结构化输出、工具调用和 Agent 循环。
3. 独立实现工具型 Agent、RAG Agent、工作流 Agent，理解多 Agent 的使用边界。
4. 设计工具 schema、状态、记忆、重试、超时、幂等、权限和人工审批。
5. 建立离线评估集，测量任务成功率、工具选择正确率、答案忠实度、延迟和成本。
6. 用 Docker、日志、追踪、CI 和基础云服务部署并排障。
7. 提供两个可运行作品、README、架构图、测试、演示视频和复盘材料。

现实提醒：任何教程都不能保证录用。求职结果还取决于编码基础、项目质量、表达、地区与岗位需求。本教程提供的是一套面向岗位能力的训练路径。

1. Agent 工程师到底做什么

Agent 是能够围绕目标，在运行循环中使用模型、工具、状态与控制逻辑完成多步骤任务的应用。工程岗位关注的不是“让模型更会聊天”，而是让系统在真实约束下稳定完成任务。

1.1 典型工作内容

- 接入模型 API，设计结构化输入输出与降级策略。
- 把数据库、搜索、内部 API、文件系统封装为可审计工具。
- 编排确定性工作流与模型决策步骤。
- 构建 RAG：摄取、切分、索引、检索、重排、引用与权限过滤。
- 实现会话状态、短期记忆、长期记忆和检查点。
- 建立评测集、追踪链路、错误分类、回归测试与成本看板。
- 处理提示注入、越权工具调用、敏感数据泄露和高风险操作审批。
- 将 Agent 作为后端服务部署，并与 Web、企业系统或 MCP 工具连接。

1.2 先工作流，后自治

如果步骤固定、规则明确，应优先编写普通代码或工作流。只有当任务路径无法完全预先枚举、需要模型在运行时选择工具或分解任务时，才增加 Agent 自主性。生产系统通常是“确定性外壳 + 有限 Agent 决策”，不是无限循环的全自动机器人。

2. 入学诊断与补课清单

请在开始前完成下列自测。低于 70 分时，先用 1-2 周补基础。

Python	函数、类、类型注解、异常、上下文管理器、asyncio	实现异步并发请求器，含超时与重试
数据结构	列表、字典、堆、图、复杂度	手写 BFS/DFS、LRU 缓存
后端	HTTP、REST、鉴权、数据库、缓存	写一个带 JWT 与分页的 FastAPI CRUD
工程化	Git、pytest、日志、环境变量	PR 流程、单测、lint、配置隔离
Linux/Docker	shell、进程、端口、镜像、容器	Docker 化 FastAPI + PostgreSQL
数据库	SQL、索引、事务	解释慢查询并添加合适索引
英文文档	能读 API 文档和报错	独立按官方 quickstart 跑通示例

2.1 推荐开发环境

- Python 3.11 或当前项目支持的稳定版本
- uv 或 Poetry 管理依赖
- VS Code / PyCharm, Git, Docker Desktop
- PostgreSQL, Redis（按项目需要）
- pytest、ruff、mypy 或 pyright、pre-commit
- 任一支持工具调用与结构化输出的模型服务

建议目录：

```
agent-app/  
├── app/  
│   ├── api/           # HTTP 接口  
│   ├── agents/        # Agent 逻辑  
│   ├── tools/         # 工具 schema  
│   ├── rag/           # 检索增强生成  
│   ├── domain/        # 领域模型  
│   ├── infra/         # DB 基础设施  
│   ├── observability/ # 可观测性  
│   └── config.py  
├── tests/  
│   ├── unit/  
│   ├── integration/  
│   └── evals/  
├── scripts/  
├── docs/  
├── .env.example  
├── pyproject.toml  
├── Dockerfile  
└── README.md
```

3. 16 周上岗路线图

工程底座	1-2	FastAPI 服务、测试、Docker	CI 通过，接口有错误处理
LLM 基础	3-4	结构化抽取、工具调用	输出可验证，失败可重试
Agent 核心	5-6	手写循环、单 Agent	可控终止，工具可审计
RAG	7-8	带引用的知识助手	有评测集与检索指标
状态与编排	9-10	LangGraph/SDK 工作流	可恢复、可人工审批
生产化	11-12	安全、追踪、成本、部署	可观察、可回滚、有限流
作品集	13-15	两个完整项目	文档、测试、演示齐全
求职冲刺	16	简历、面试题、项目答辩	30 分钟讲清架构与取舍

4. 第 1-2 周：后端与工程化底座

4.1 必学知识

- HTTP 方法、状态码、鉴权、CORS、SSE/WebSocket 的区别。
- Pydantic 模型、依赖注入、统一异常、OpenAPI 文档。
- async/await、连接池、超时、指数退避、熔断概念。
- PostgreSQL 表设计、索引、事务；Redis 缓存与分布式锁基本边界。
- 单元、集成、端到端测试分层；测试替身与依赖注入。
- Docker 多阶段构建、健康检查、非 root 用户、环境变量。

4.2 必做练习：任务管理 API

实现用户注册、登录、任务 CRUD、标签、分页、审计日志。

验收：

- 请求模型和响应模型分离。
- 所有外部 I/O 有超时。
- 重要写操作具备幂等键或重复提交保护。
- pytest 覆盖核心业务规则，而非只测 200 状态码。
- `.env.example` 不包含真实密钥。
- Docker Compose 一条命令启动 API 与数据库。

5. 第 3-4 周：LLM 应用基础

5.1 你必须真正理解的概念

消息与指令层级：把稳定行为放在系统级指令，把任务数据放在用户输入或工具结果中。不要把不可信网页内容拼成高权限指令。

结构化输出：让模型返回受 schema 约束的对象，再由程序验证。模型生成 JSON 不等于 JSON 一定合法，也不等于语义正确。

工具调用：模型只负责提出调用意图和参数。真正的执行、鉴权、参数校验、超时和审计属于应用代码。

上下文工程：选择哪些指令、历史、检索片段、工具说明进入当前上下文。目标是相关、最小、可追踪，而非越多越好。

错误分类：至少区分网络错误、限流、模型拒绝、schema 失败、工具业务错误、权限错误、超时和质量不合格。

5.2 第一个结构化抽取服务

任务：输入招聘 JD，输出岗位名称、技能、经验、职责、加分项与缺失字段。

伪代码：

```
class JobProfile(BaseModel):
    title: str
    must_have_skills: list[str]
    preferred_skills: list[str]
    years_experience: int | None
    responsibilities: list[str]
    missing_information: list[str]

async def extract_job(text: str) -> JobProfile:
    raw = await model.generate(
        instructions="□□□□□□□□□□□□□□ missing_information",
        input=text,
        output_schema=JobProfile.model_json_schema(),
        timeout=30,
    )
    return JobProfile.model_validate(raw)
```

验收：准备 30 条不同格式 JD，记录 schema 成功率、字段准确性、平均延迟、失败原因。不要只做 3 条演示。

6. 第 5-6 周：Agent 核心机制

6.1 手写一个最小 Agent 循环

先手写，再使用框架。你要掌握循环，而不是背 API。

```
async def run_agent(user_input, tools, max_steps=8):
    state = [{"role": "user", "content": user_input}]
    for step in range(max_steps):
        decision = await call_model(state, tool_schemas(tools))
        if decision.type == "final":
            return decision.output
        if decision.type != "tool_call":
            raise RuntimeError("unknown decision")
        tool = tools.get(decision.name)
        args = tool.input_model.model_validate(decision.arguments)
        result = await tool.execute(args, timeout=tool.timeout)
        state.append(decision.as_message())
        state.append(sanitize_tool_result(result))
    raise StepLimitExceeded(max_steps)
```

生产版还应具备：

- 总步骤、总耗时、总 token 或费用预算。
- 工具白名单与基于用户身份的权限判断。
- 重复调用检测，避免同参数无限循环。
- 读操作与写操作分级，写操作前人工审批。
- 工具结果大小限制与敏感字段脱敏。
- 全链路 trace_id、步骤日志和最终状态。

6.2 工具设计规范

一个好工具应当单一职责、名称明确、参数少而强类型、返回结构稳定、错误可分类。

```
class SearchOrdersInput(BaseModel):
    customer_id: str
    status: Literal["pending", "paid", "shipped", "cancelled"] | None = None
    limit: int = Field(default=20, ge=1, le=100)

class SearchOrdersResult(BaseModel):
    items: list[OrderSummary]
    next_cursor: str | None
```

避免：run_sql(query: str)、call_any_api(url, body) 这类权限面过大的通用工具。

6.3 单 Agent 项目：个人学习教练

工具：课程查询、任务创建、学习记录、复习计划。要求：

- Agent 不能直接修改历史记录，修改需要确认。
- 计划必须输出结构化任务。
- 日程冲突由普通代码检测。
- 所有写操作记录 who/when/what/before/after。

7. 第 7-8 周：RAG 从“能搜”到“可信”

7.1 标准流水线

1. 摄取：解析 PDF、Markdown、网页或数据库记录。
2. 清洗：去页眉页脚、重复段、乱码和无意义导航。
3. 切分：按语义结构和标题切分，保留文档、章节、页码等元数据。
4. 嵌入与索引：建立向量索引；必要时加入关键词索引。
5. 检索：查询改写、过滤、混合检索、重排。
6. 生成：只使用证据回答，输出引用，证据不足时明确说明。
7. 评估：检索命中与回答质量分开测。

7.2 不要迷信固定 chunk 大小

切分策略应由文档结构、问答粒度和模型限制决定。实验至少比较：

- 标题感知切分与固定长度切分。
- 纯向量与关键词+向量混合检索。
- 不重排与重排。
- top-k 不同取值。

7.3 评估指标

- Recall@k：标准证据是否出现在前 k 个结果。
- MRR：首个相关证据的排名质量。
- 引用正确率：引用是否真的支持答案。

- 忠实度：答案陈述能否由上下文支持。
- 拒答合理性：无证据时是否避免编造。
- 端到端任务成功率、P50/P95 延迟与单次成本。

7.4 RAG 项目验收

建立至少 80 条人工检查过的问题集，覆盖事实查询、多跳组合、表格、否定问题、无答案、权限文档。每次修改切分、提示、模型或检索策略都跑回归测试，并保存版本与结果。

8. 第 9-10 周：状态、记忆与编排框架

8.1 三类状态

- 运行状态：当前步骤、工具结果、预算、错误和审批状态。
- 会话记忆：当前会话需要保留的历史摘要与用户目标。
- 长期记忆：跨会话偏好或事实，必须有来源、更新规则、删除机制与权限。

不要把完整聊天历史无限塞进上下文。采用窗口、摘要和按需检索，并监控摘要失真。

8.2 LangGraph 学习重点

LangGraph 适合需要显式状态、图结构、持久化、流式和人工介入的长流程。学习顺序：State → Node → Edge → Conditional Edge → Checkpoint → Interrupt → Streaming → Subgraph → Testing。

练习：做一个“研究报告工作流”：需求澄清 → 计划 → 并行检索 → 证据检查 → 大纲 → 草稿 → 引用核对 → 人工审批 → 导出。

8.3 SDK 选型原则

- 只需一个轻量级 Agent：优先模型官方 SDK 或较薄的 Agent SDK。
- 需要明确状态机、恢复和人工审批：考虑 LangGraph 或同类图编排。
- 需要微软生态的新项目：优先评估 Microsoft Agent Framework。AutoGen 的仓库说明已建议新用户转向其后继框架。
- 团队已有成熟框架：统一日志、评测和部署比追逐新框架更重要。

不要在一个作品中同时堆三个框架。面试官更关心你为什么选、如何测试和如何处理失败。

9. 第 11-12 周：生产化能力

9.1 可靠性

- 所有调用设置 connect/read/total timeout。
- 仅对可重试错误进行有限次数指数退避，并加入抖动。
- 写操作使用幂等键，重试前确认副作用。
- 对模型、工具、检索分别建立降级方案。
- 设置最大步骤、最大并发、最大上下文、预算上限。

- 失败返回可操作错误，不把内部堆栈暴露给用户。

9.2 可观测性

每次运行至少记录：trace_id、用户与租户的匿名标识、模型、提示版本、工具名、工具耗时、状态转换、token、费用估算、检索文档 ID、错误分类、最终结果与人工反馈。敏感数据按策略脱敏。

关键看板：

- 请求量、成功率、任务成功率。
- P50/P95/P99 延迟。
- 工具错误率、超时率、重试率。
- 单任务 token 与成本。
- 无答案率、人工接管率、用户反馈。

9.3 安全红线

- 把模型输出视为不可信输入。
- 工具参数必须 schema 校验，服务端再次鉴权。
- 外部网页、文件和工具结果不可提升为系统指令。
- 数据按租户/用户隔离，检索前做权限过滤，而不是生成后过滤。
- 高风险写操作采用确认页、审批或双人复核。
- 密钥只放密钥管理系统或环境变量，不进代码、日志和提示。
- 对代码执行、文件访问和网络访问使用沙箱与最小权限。
- 建立审计、撤销、数据保留与删除机制。

9.4 部署清单

- [] Docker 镜像固定依赖版本并通过漏洞扫描
- [] 服务以非 root 用户运行
- [] 健康检查区分 liveness/readiness
- [] 数据库迁移可回滚
- [] CI 执行 lint、类型检查、单测、集成测试与评测门禁
- [] staging 使用脱敏数据
- [] 灰度发布，关键指标异常自动回滚
- [] 提示词、工具 schema 与评测集都有版本号

10. MCP：把工具接入做成标准接口

MCP 用于在 LLM 应用与外部数据/工具之间建立标准化连接。学习时重点理解 host、client、server、tools、resources、prompts、能力协商、传输与授权，而不是只会启动一个 demo server。

练习：实现一个只读“课程资料 MCP Server”：

- `search_course_notes(query, course_id)`

- `get_note(note_id)`
- `list_assignments(course_id)`

安全要求：只读、参数校验、返回大小限制、每次调用审计、资源级权限。完成后再增加一个需要人工确认的写工具 `create_study_task`。

11. 两个作品集项目

项目 A：企业知识与工单助手

问题：客服需要在知识库、历史工单和订单工具之间切换。

功能：用户鉴权、混合检索、引用回答、订单只读查询、工单草稿、人工确认后创建、反馈闭环。建议架构：FastAPI + PostgreSQL + 向量库 + Redis + Agent 编排 + 追踪平台。必须展示：

- 权限过滤发生在检索阶段。
- 工具调用的 schema、超时、幂等与审计。
- 100 条以上评测用例，分质量、可靠性和安全三组。
- 提示注入测试与越权测试。
- Docker Compose、本地演示数据、架构图与 ADR。

项目 B：代码仓库维护 Agent

功能：读取 issue，检索仓库，生成变更计划，得到批准后建立分支，修改受限目录，运行测试，生成 PR 描述。边界：默认只读；写文件、执行命令、提交代码都要策略检查；禁止访问密钥文件；限制命令白名单、目录和执行时间。

评测：准备 30 个可重复 issue，检查计划质量、修改正确率、测试通过率、误改文件数、平均步骤与失败恢复。

项目 README 模板

```
# 项目概述
## 1. 项目背景
## 2. 项目目标
## 3. 项目范围
## 4. 项目结构
## 5. 项目依赖
## 6. 项目安装
## 7. 项目运行
## 8. 项目测试
## 9. 项目部署
## 10. 项目维护
```

12. 测试与评测实战

12.1 测试金字塔

- 单元测试：工具参数、业务规则、状态转移、提示渲染、权限函数。
- 集成测试：模型适配器、数据库、向量库、MCP server、外部 API mock。
- 契约测试：工具 schema、返回字段与错误码兼容性。
- 评测：固定数据集上测质量、任务完成和安全。
- 端到端：少量真实流程，验证主链路。

12.2 评测数据结构

```
{
  "case_id": "rag-042",
  "input": {"question": "...", "user_role": "student"},
  "expected": {
    "must_include": [...],
    "evidence_ids": ["doc-12#p3"],
    "allowed_tools": ["search_notes"],
    "forbidden_tools": ["delete_note"]
  },
  "tags": ["multi_hop", "permission"],
  "reviewer": "human"
}
```

12.3 质量门禁建议

每次合并前比较基线。若任务成功率下降、关键安全用例失败、P95 延迟或成本超过团队阈值，则阻止发布。阈值应由项目基线和业务风险决定，不要照搬他人数字。

13. 成本与性能优化顺序

- 1. 先减少无效步骤和重复工具调用。
- 2. 缩短上下文，只保留相关证据与摘要。
- 3. 对稳定前缀、嵌入和检索结果做合适缓存。
- 4. 简单分类、改写、抽取使用更小模型，并用评测验证。
- 5. 并行执行相互独立的只读工具。
- 6. 设置早停与预算，避免失败任务持续消耗。
- 7. 最后再做复杂路由，不要过早优化。

14. 常见失败模式与排查

工具选错	描述重叠、工具过多、示例误导	trace → schema → 描述 → 负例
参数经常缺失	schema 太宽、提示不明确	验证错误 → 字段说明 → 澄清流程
Agent 死循环	无步数限制、结果未改变	重复调用检测 → 状态 → 终止条件
RAG 有引用但答错	检索错或引用未核验	Recall@k → 重排 → 生成忠实度
偶发重复写入	重试与副作用冲突	幂等键 → 事务 → 审计日志
延迟突然升高	串行工具、上下文膨胀	分段耗时 → token → 并发与缓存
线上无法复现	缺少版本与 trace	提示/模型/工具版本 → 输入快照

15. 面试准备

15.1 高频问题

- 1. Agent 与固定工作流有什么区别？何时不该用 Agent？
- 2. 你如何限制 Agent 的步骤、费用与权限？

- 3. 工具调用失败后怎样重试而不产生重复副作用？
- 4. RAG 的检索质量与生成质量如何分别评估？
- 5. 如何防止提示注入导致越权工具调用？
- 6. 短期记忆、长期记忆、运行状态分别怎么存？
- 7. 为什么选 LangGraph/某 SDK，而不是手写循环？
- 8. 如何设计可恢复的人机审批流程？
- 9. 线上质量下降时，你先看哪些指标和 trace？
- 10. 如何让评测成为 CI 门禁？

15.2 项目答辩的 8 分钟结构

- 1 分钟：业务问题、用户和成功标准。
- 1 分钟：为什么需要 Agent，哪些部分仍是确定性代码。
- 2 分钟：架构、状态、工具与一次请求时序。
- 1 分钟：RAG/记忆/权限的关键设计。
- 1 分钟：测试、评测数据与结果。
- 1 分钟：故障、安全与成本。
- 1 分钟：失败案例、取舍与下一步。

16. 简历表达模板

不要写“学习了 LangChain，调用了大模型”。使用“行动 + 规模 + 方法 + 结果”，只填写你真实测量的数据。

```

00000000  ____ 000  ____ Agent000  ____ 00000000
00  ____ 00000000 CI 00000000000000  ____ 000  ____
00  ____  ____  ____ 00 P95 00/000000
000000000000000000000000  ____ 00000000

```

17. 每周执行 SOP

周一：确定本周一个可演示产出与验收标准。周二至周四：每天 60% 编码、20% 测试、10% 文档、10% 复盘。
周五：跑完整评测，修复最高频错误。周六：录 5 分钟演示，更新 README 和架构决策记录。
周日：复盘、整理面试题、安排下周；保留半天休息。

每日结束回答：

- 1. 今天提交了什么可验证产物？
- 2. 哪个失败最有信息量？
- 3. 证据是什么：测试、trace、指标还是用户反馈？
- 4. 明天最小下一步是什么？

18. 结业验收清单

编码与架构

- [] 能在 60 分钟内写出带测试的工具调用 Agent。

- [] 能解释状态、循环、检查点、人工审批和终止条件。
- [] 工具强类型、最小权限、超时、错误分类、幂等、审计齐全。

RAG 与评测

- [] 能建立摄取到引用的完整链路。
- [] 有人工审查评测集，区分检索与生成问题。
- [] 能展示至少一次失败驱动和改进实验。

生产化

- [] Docker 一键启动，README 可被同学复现。
- [] 有结构化日志、trace、核心指标和告警思路。
- [] 有安全测试、限流、预算和降级策略。

求职材料

- [] 两个项目都有在线或本地演示、架构图、测试和评测报告。
- [] 简历不写虚假指标，所有数字可追溯。
- [] 能完成 30 分钟项目深挖与白板设计。

19. 官方资料与学习顺序

1. OpenAI Agents SDK：从 quickstart、Agent 定义、运行、工具、编排、guardrails、tracing 和 evals 依次学习。

<https://developers.openai.com/api/docs/guides/agents>

2. LangGraph：重点学习状态、持久化、流式、人机介入和确定性步骤与 Agent 步骤混合。

<https://docs.langchain.com/oss/python/langgraph/overview>

3. Microsoft Agent Framework：微软生态新项目应评估该框架；AutoGen 仓库已说明新用户应转向其后继框架。

<https://learn.microsoft.com/zh-cn/agent-framework/>

4. AutoGen AgentChat：可用于理解多 Agent 高级 API，但使用前先确认项目维护状态与迁移路径。

<https://microsoft.github.io/autogen/stable/user-guide/agentchat-user-guide/index.html>

5. MCP 规范：学习 tools/resources/prompts、协议、授权及安全边界。

<https://modelcontextprotocol.io/specification/>

框架 API 更新很快。项目中锁定版本，阅读对应版本的官方文档，并在升级时跑完整回归评测。

附录 A：16 周逐周任务卡

第 1 周：工程环境与 Python 复习

本周产出：异步请求器、类型检查、pytest 过关条件：并发 20 个 mock 请求，超时与重试可测

- 学习：阅读对应官方文档并写一页概念笔记。
- 编码：至少 5 次小提交，每次提交只解决一个问题。
- 测试：为成功、失败、边界、权限各补用例。
- 评测：保存基线结果，不凭主观感觉宣布“优化”。
- 文档：更新启动命令、架构变化、已知限制。
- 复盘：记录一次失败、根因、证据与预防措施。

第 2 周：FastAPI 与 Docker

本周产出：带鉴权的任务 API 过关条件：Compose 启动，CI 通过

- 学习：阅读对应官方文档并写一页概念笔记。
- 编码：至少 5 次小提交，每次提交只解决一个问题。
- 测试：为成功、失败、边界、权限各补用例。
- 评测：保存基线结果，不凭主观感觉宣布“优化”。
- 文档：更新启动命令、架构变化、已知限制。
- 复盘：记录一次失败、根因、证据与预防措施。

第 3 周：模型 API 与结构化输出

本周产出：JD 抽取服务 过关条件：30 条样本，记录失败类型

- 学习：阅读对应官方文档并写一页概念笔记。
- 编码：至少 5 次小提交，每次提交只解决一个问题。
- 测试：为成功、失败、边界、权限各补用例。
- 评测：保存基线结果，不凭主观感觉宣布“优化”。
- 文档：更新启动命令、架构变化、已知限制。
- 复盘：记录一次失败、根因、证据与预防措施。

第 4 周：工具调用与上下文工程

本周产出：天气/课程/任务工具集 过关条件：参数验证、错误分类、trace

- 学习：阅读对应官方文档并写一页概念笔记。
- 编码：至少 5 次小提交，每次提交只解决一个问题。
- 测试：为成功、失败、边界、权限各补用例。

- 评测：保存基线结果，不凭主观感觉宣布“优化”。
- 文档：更新启动命令、架构变化、已知限制。
- 复盘：记录一次失败、根因、证据与预防措施。

第 5 周：手写 Agent 循环

本周产出：有限步骤 Agent 过关条件：终止、重复调用检测、预算

- 学习：阅读对应官方文档并写一页概念笔记。
- 编码：至少 5 次小提交，每次提交只解决一个问题。
- 测试：为成功、失败、边界、权限各补用例。
- 评测：保存基线结果，不凭主观感觉宣布“优化”。
- 文档：更新启动命令、架构变化、已知限制。
- 复盘：记录一次失败、根因、证据与预防措施。

第 6 周：单 Agent 项目

本周产出：学习教练 v1 过关条件：写操作需确认，有审计

- 学习：阅读对应官方文档并写一页概念笔记。
- 编码：至少 5 次小提交，每次提交只解决一个问题。
- 测试：为成功、失败、边界、权限各补用例。
- 评测：保存基线结果，不凭主观感觉宣布“优化”。
- 文档：更新启动命令、架构变化、已知限制。
- 复盘：记录一次失败、根因、证据与预防措施。

第 7 周：RAG 摄取与检索

本周产出：课程资料索引 过关条件：元数据、权限、Recall@k

- 学习：阅读对应官方文档并写一页概念笔记。
- 编码：至少 5 次小提交，每次提交只解决一个问题。
- 测试：为成功、失败、边界、权限各补用例。
- 评测：保存基线结果，不凭主观感觉宣布“优化”。
- 文档：更新启动命令、架构变化、已知限制。
- 复盘：记录一次失败、根因、证据与预防措施。

第 8 周：RAG 生成与评测

本周产出：带引用问答 过关条件：80 条评测与拒答测试

- 学习：阅读对应官方文档并写一页概念笔记。
- 编码：至少 5 次小提交，每次提交只解决一个问题。

- 测试：为成功、失败、边界、权限各补用例。
- 评测：保存基线结果，不凭主观感觉宣布“优化”。
- 文档：更新启动命令、架构变化、已知限制。
- 复盘：记录一次失败、根因、证据与预防措施。

第 9 周：LangGraph 状态图

本周产出：研究工作流 过关条件：条件边、检查点、恢复

- 学习：阅读对应官方文档并写一页概念笔记。
- 编码：至少 5 次小提交，每次提交只解决一个问题。
- 测试：为成功、失败、边界、权限各补用例。
- 评测：保存基线结果，不凭主观感觉宣布“优化”。
- 文档：更新启动命令、架构变化、已知限制。
- 复盘：记录一次失败、根因、证据与预防措施。

第 10 周：记忆与人工介入

本周产出：可编辑长期记忆 过关条件：来源、删除、审批

- 学习：阅读对应官方文档并写一页概念笔记。
- 编码：至少 5 次小提交，每次提交只解决一个问题。
- 测试：为成功、失败、边界、权限各补用例。
- 评测：保存基线结果，不凭主观感觉宣布“优化”。
- 文档：更新启动命令、架构变化、已知限制。
- 复盘：记录一次失败、根因、证据与预防措施。

第 11 周：可靠性与安全

本周产出：故障注入套件 过关条件：超时、重试、幂等、注入

- 学习：阅读对应官方文档并写一页概念笔记。
- 编码：至少 5 次小提交，每次提交只解决一个问题。
- 测试：为成功、失败、边界、权限各补用例。
- 评测：保存基线结果，不凭主观感觉宣布“优化”。
- 文档：更新启动命令、架构变化、已知限制。
- 复盘：记录一次失败、根因、证据与预防措施。

第 12 周：可观测与部署

本周产出：看板和 staging 过关条件：分段延迟、成本、回滚

- 学习：阅读对应官方文档并写一页概念笔记。

- 编码：至少 5 次小提交，每次提交只解决一个问题。
- 测试：为成功、失败、边界、权限各补用例。
- 评测：保存基线结果，不凭主观感觉宣布“优化”。
- 文档：更新启动命令、架构变化、已知限制。
- 复盘：记录一次失败、根因、证据与预防措施。

第 13 周：项目 A 冲刺

本周产出：知识工单助手 过关条件：主链路可演示

- 学习：阅读对应官方文档并写一页概念笔记。
- 编码：至少 5 次小提交，每次提交只解决一个问题。
- 测试：为成功、失败、边界、权限各补用例。
- 评测：保存基线结果，不凭主观感觉宣布“优化”。
- 文档：更新启动命令、架构变化、已知限制。
- 复盘：记录一次失败、根因、证据与预防措施。

第 14 周：项目 A 评测与文档

本周产出：评测报告、ADR 过关条件：失败案例可复现

- 学习：阅读对应官方文档并写一页概念笔记。
- 编码：至少 5 次小提交，每次提交只解决一个问题。
- 测试：为成功、失败、边界、权限各补用例。
- 评测：保存基线结果，不凭主观感觉宣布“优化”。
- 文档：更新启动命令、架构变化、已知限制。
- 复盘：记录一次失败、根因、证据与预防措施。

第 15 周：项目 B 最小闭环

本周产出：代码维护 Agent 过关条件：沙箱、白名单、测试

- 学习：阅读对应官方文档并写一页概念笔记。
- 编码：至少 5 次小提交，每次提交只解决一个问题。
- 测试：为成功、失败、边界、权限各补用例。
- 评测：保存基线结果，不凭主观感觉宣布“优化”。
- 文档：更新启动命令、架构变化、已知限制。
- 复盘：记录一次失败、根因、证据与预防措施。

第 16 周：求职冲刺

本周产出：简历、答辩、模拟面试 过关条件：作品可复现，30 分钟讲清

- 学习：阅读对应官方文档并写一页概念笔记。
- 编码：至少 5 次小提交，每次提交只解决一个问题。
- 测试：为成功、失败、边界、权限各补用例。
- 评测：保存基线结果，不凭主观感觉宣布“优化”。
- 文档：更新启动命令、架构变化、已知限制。
- 复盘：记录一次失败、根因、证据与预防措施。

附录 B：代码审查清单

- [] 输入、工具结果、模型输出均有 schema 或显式验证。
- [] 外部 I/O 有超时，重试只覆盖可重试错误。
- [] 写操作幂等，Agent 重复调用不会重复产生副作用。
- [] 鉴权在服务端执行，不依赖模型“自觉”。
- [] 日志不包含密钥、完整个人信息或敏感文档内容。
- [] 循环有步骤、时间、费用和并发限制。
- [] 测试不依赖不稳定线上模型，核心逻辑可 mock。
- [] 评测集与提示、模型、工具 schema 一起版本化。
- [] README 的启动方式在全新环境中验证过。

附录 C：90 天后继续成长

- 深入一个方向：RAG、代码 Agent、数据分析 Agent、企业流程 Agent 或语音 Agent。
- 学习分布式任务、消息队列、Kubernetes、云鉴权和密钥管理。
- 参与开源：复现 issue、补测试、修文档，再提交小 PR。
- 每月选择一个线上失败模式，做可复现 benchmark 与技术文章。
- 关注协议和框架版本变化，但以评测和业务价值决定升级。

附录 D：Prompt 库

